# Oodles Engine 1.0 Documentation

# Introduction

**Oodles Engine** is an exciting physics-based fighting party game framework that allows players to engage in intense hand-to-hand combat, wield weapons, and manipulate physical props. The game's physics are powered by Unity's built-in physics engine.

# Table of Contents


## Part I: Play Demo

1. Movement
2. Grab Object
3. Throw
4. Attack

## Part II: Customizing Character Physics

1. Generate Characters
2. Joint Bounding Box
3. Place it in the game

## Part III: Creating Level

1. Grabbable objects
2. CustomizingWeapons
3. Customizing Game Logic

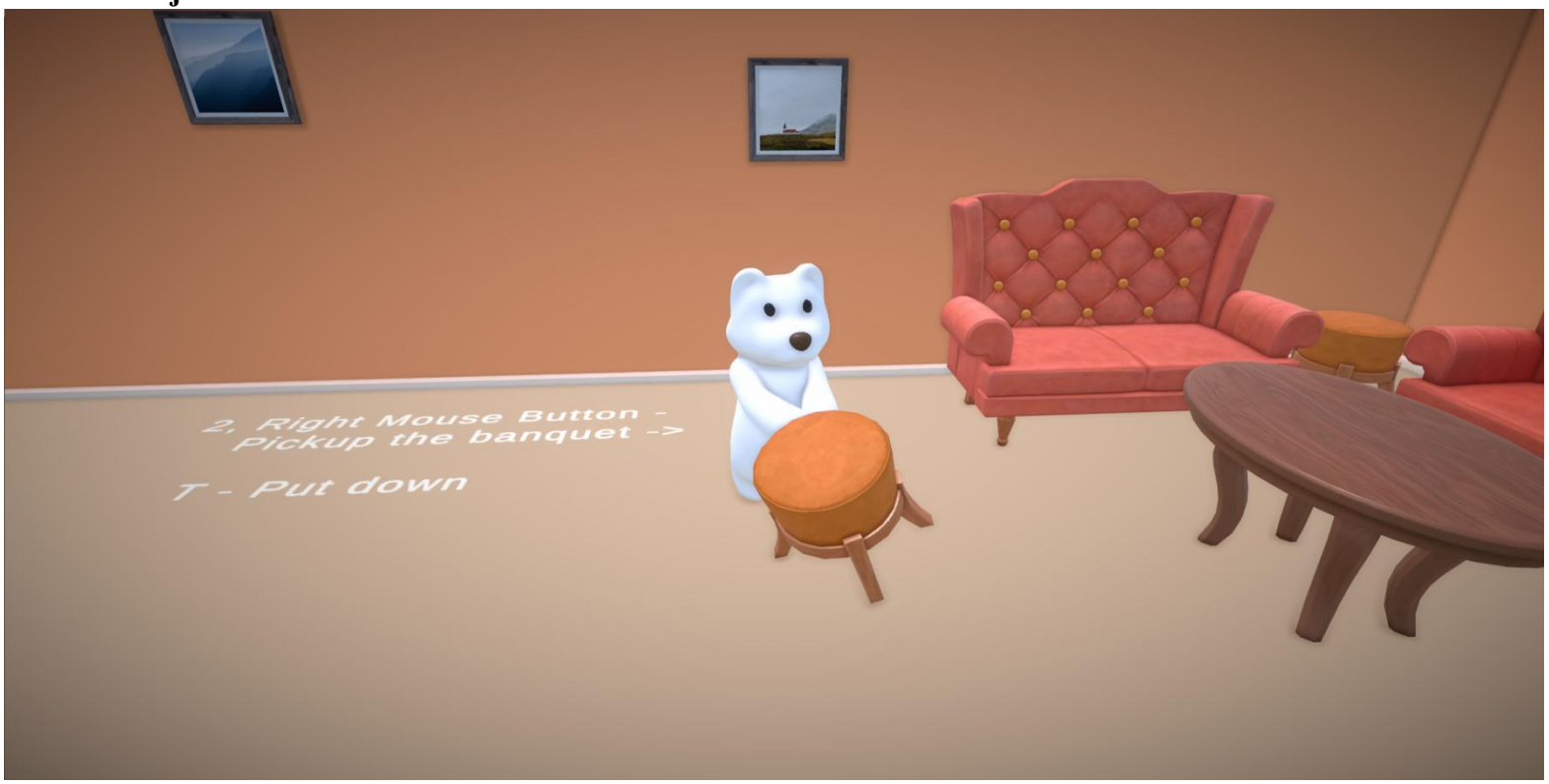
# Part I: Play Demo

Please ensure that your project settings include the tag "**CanBeGrabbed**" and the layer "**Player**", "**Ragdoll**", "**RagdollHands**". Please open **Assets/TechFusion/OodlesEngine/Demo/Scenes/TutorialRoom.unity** Scene.

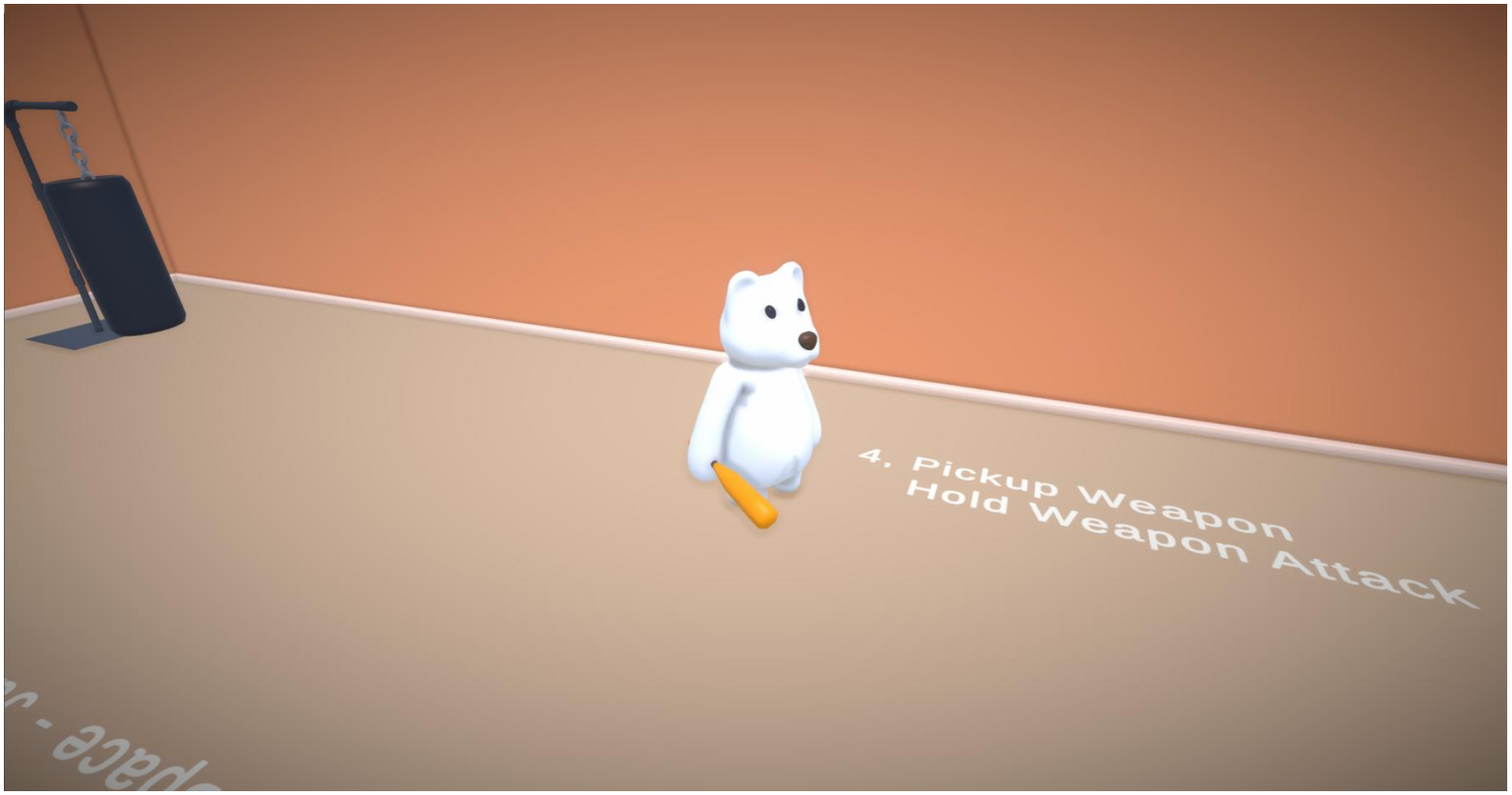
## 1. Movement

• The WASD keys control movement, and the Space key controls jumping. When energy is greater than zero, both the animation and movement speed will increase, consuming energy in the process. Once the energy is depleted, the movement will slow down.

## 2. Grab Object




• Clicking the right mouse button will pick up objects in the vicinity.

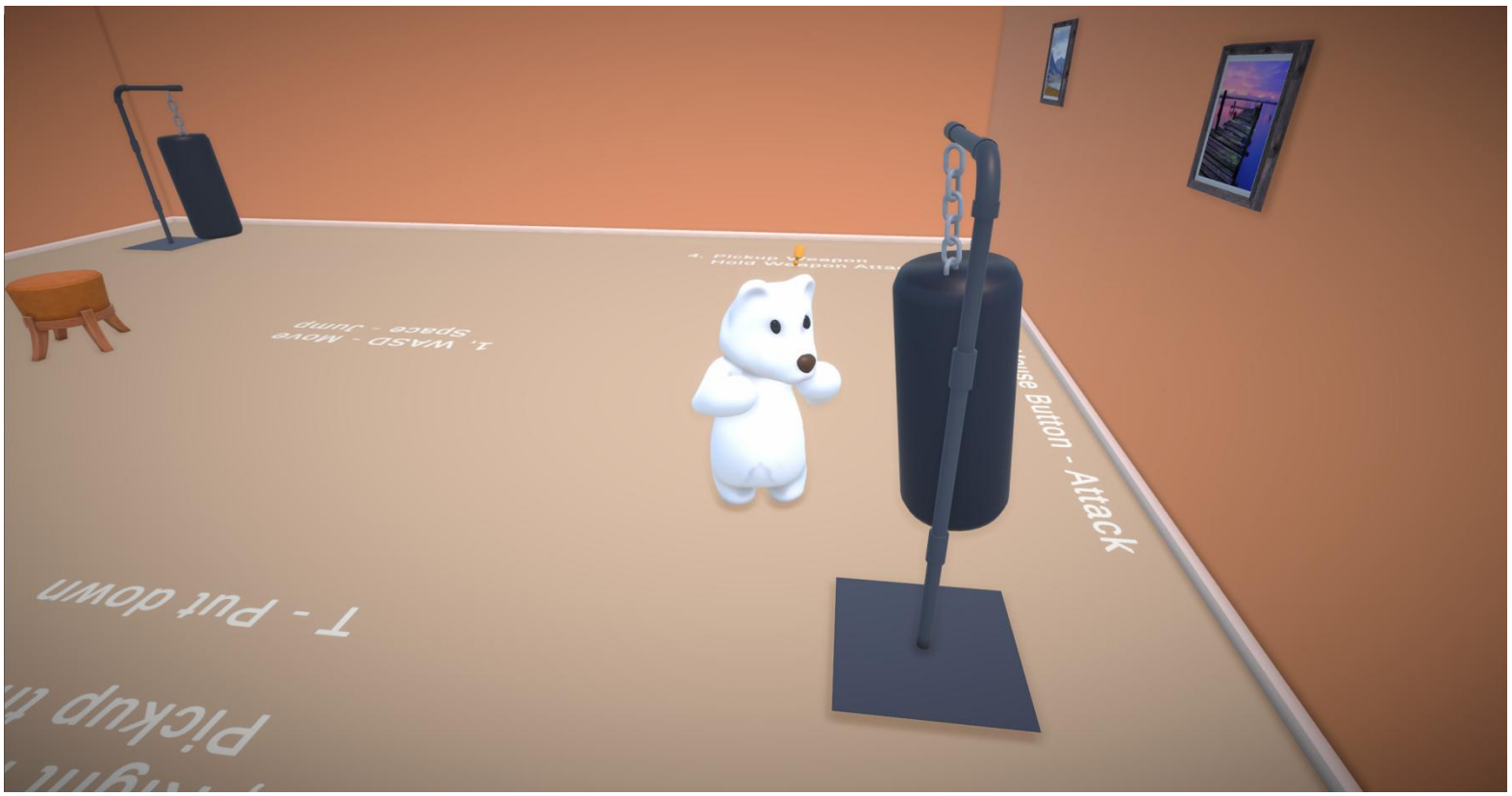



• If the object picked up is a weapon, it will automatically be equipped to the appropriate position.

## 3. Throw Object

• The 'T' key on the keyboard controls the dropping of the object in hand. If the object has a Toy component attached, it will also have an additional push force.

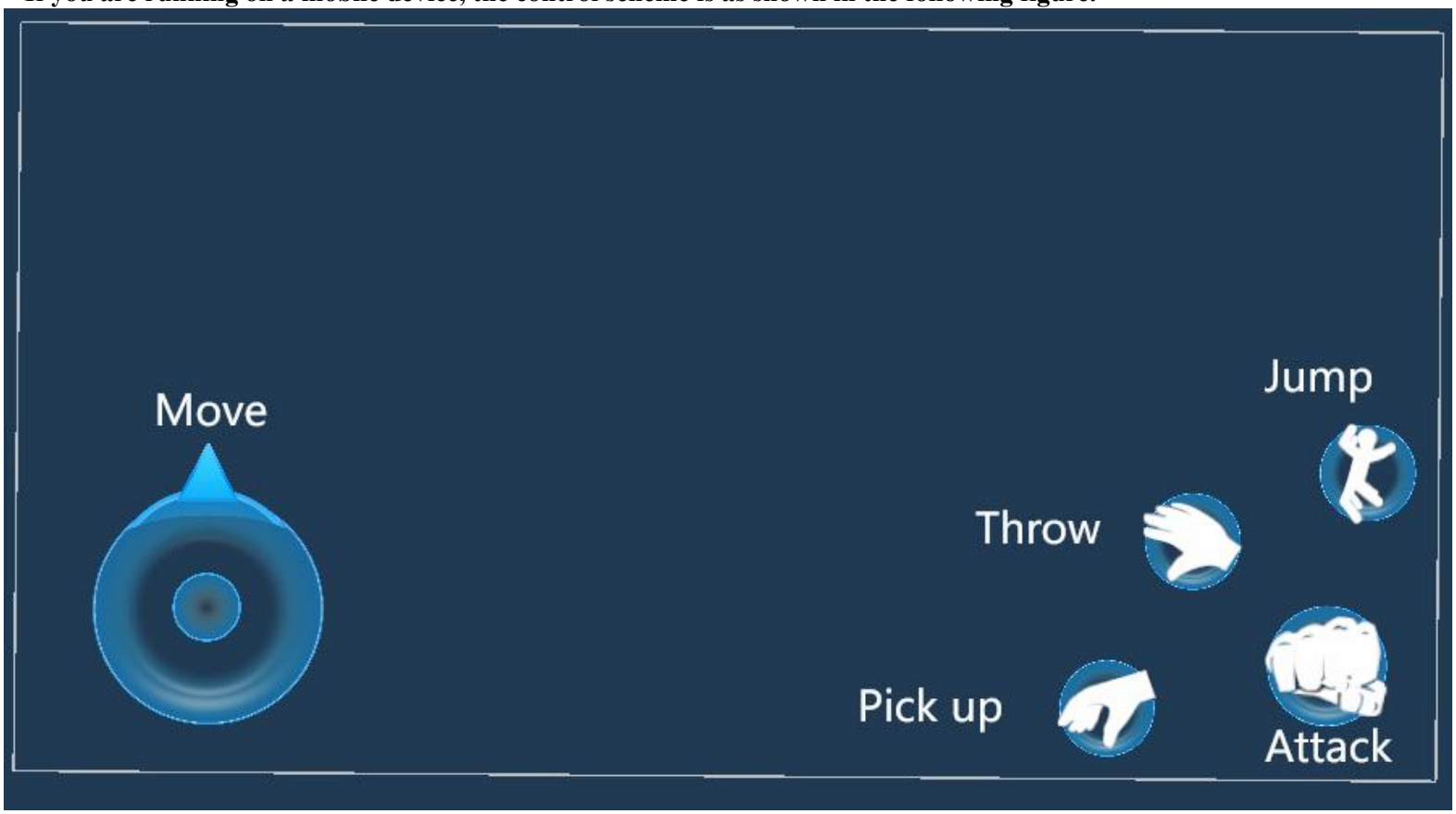
## 4. Attack




• Clicking the left mouse button will initiate an attack

• Playing the corresponding animation based on whether you are unarmed or holding a weapon.

*** If you are running on a mobile device, the control scheme is as shown in the following figure.**



# Part II: Customizing Character Physics

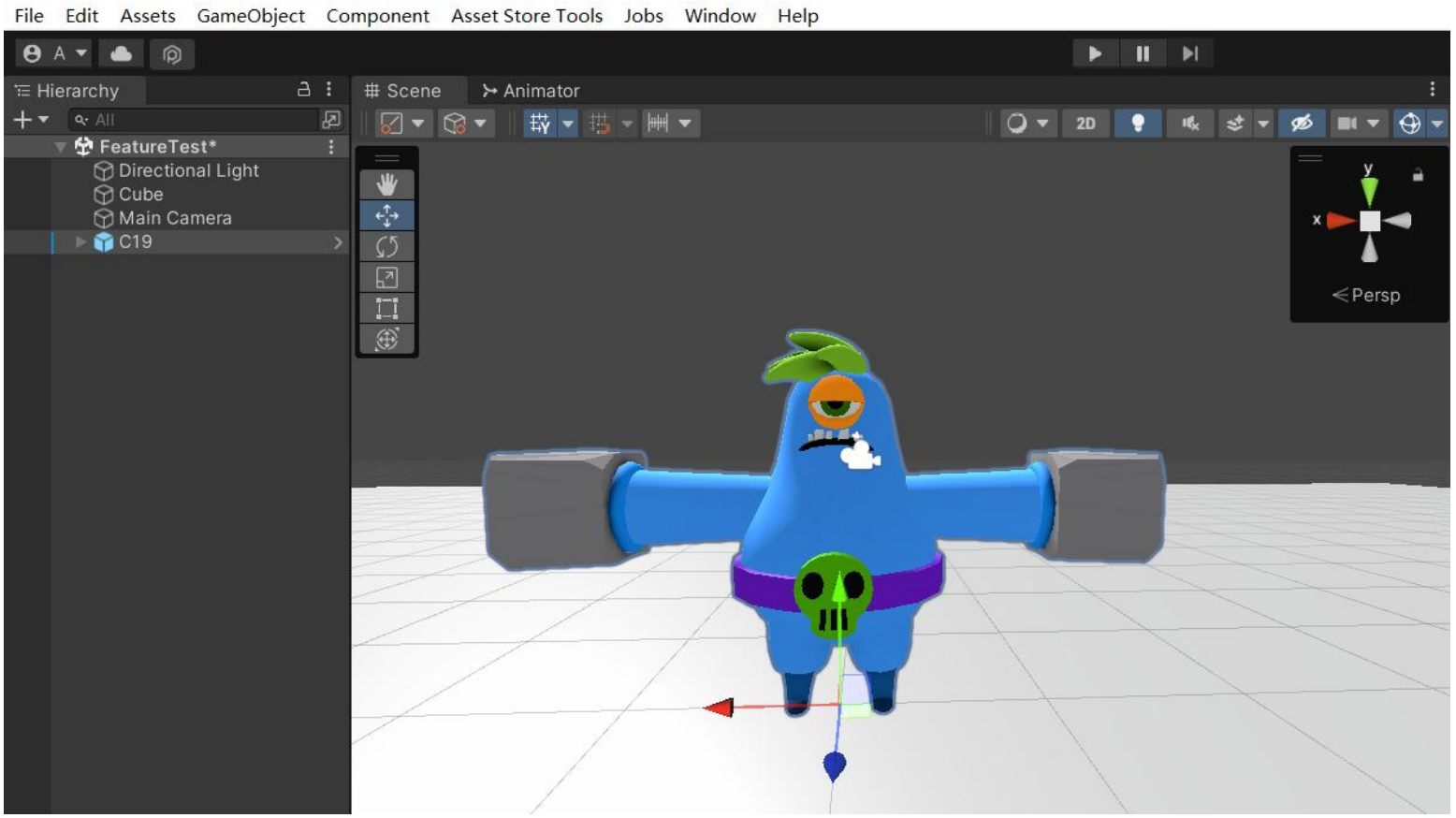
## Introduction to Active Ragdoll

**Oodles Engine** employs an active ragdoll character physics animation solution. Active ragdoll is a game animation technology used to achieve more realistic physical effects. Unlike traditional ragdoll techniques that create a limp body when a character dies, active ragdoll retains control, resulting in more natural and expressive actions.

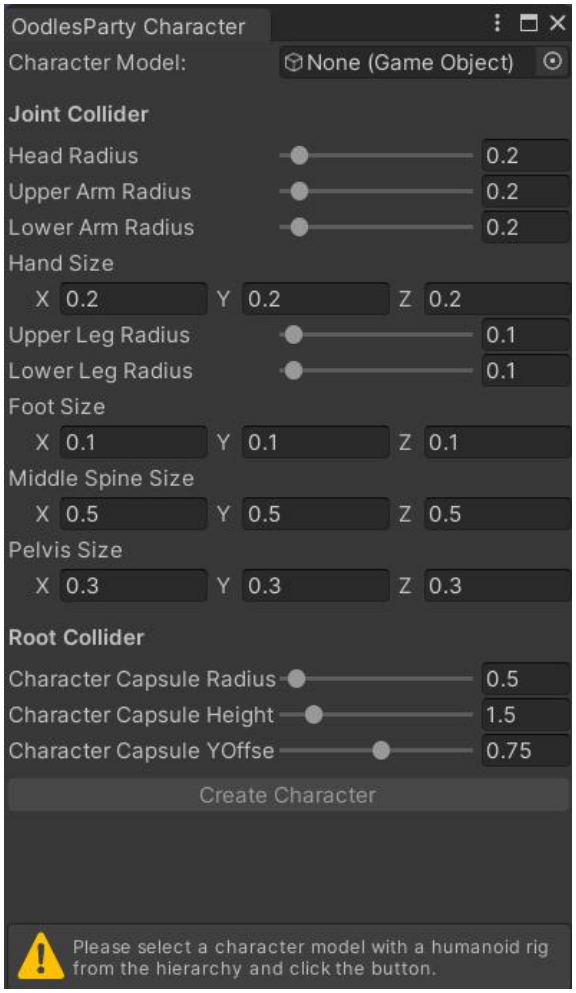
The following steps will guide you on customizing your game characters:

## 2.1 Generate Characters

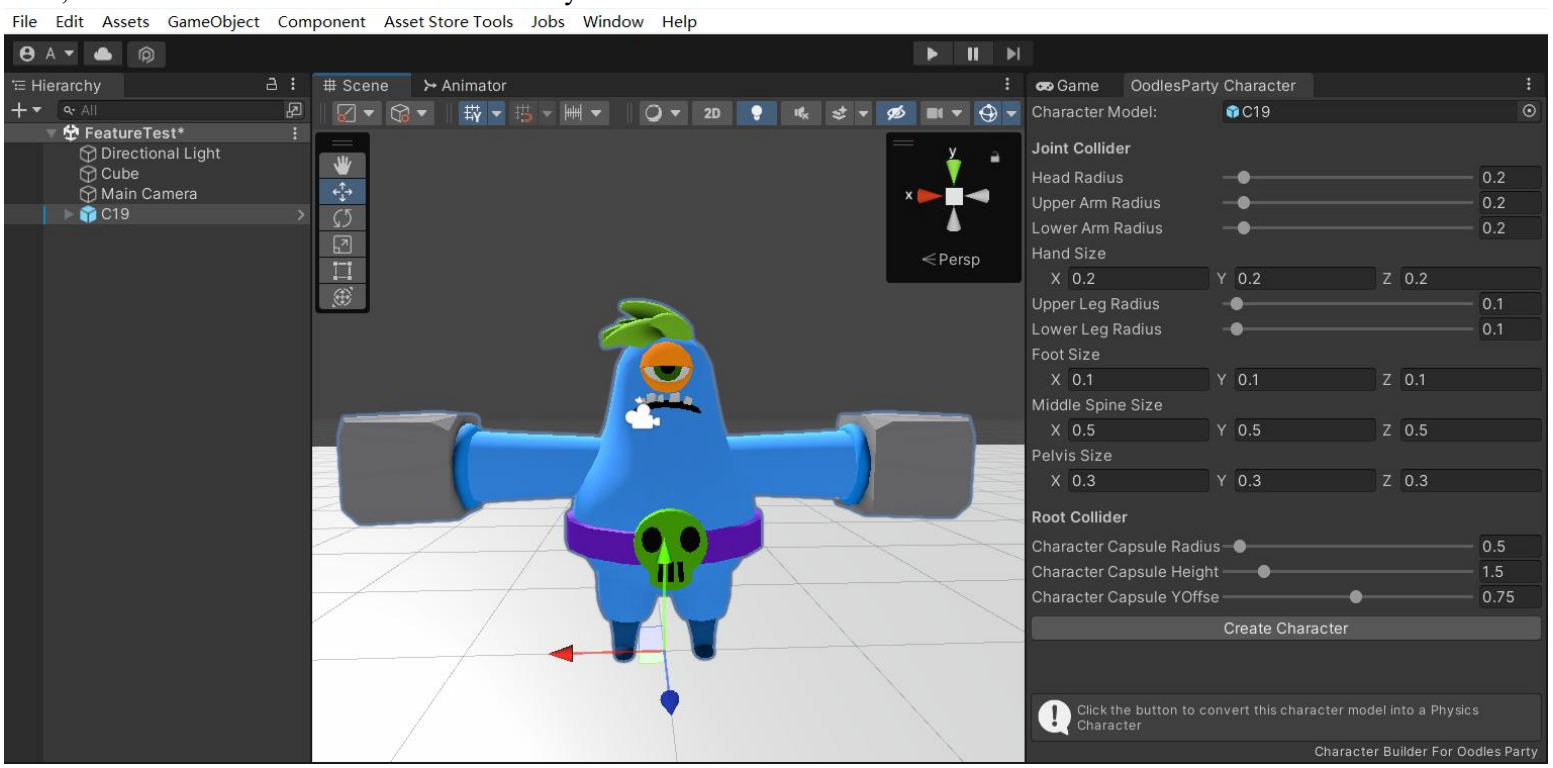
Open a new scene and place a Humanoid character model into it.



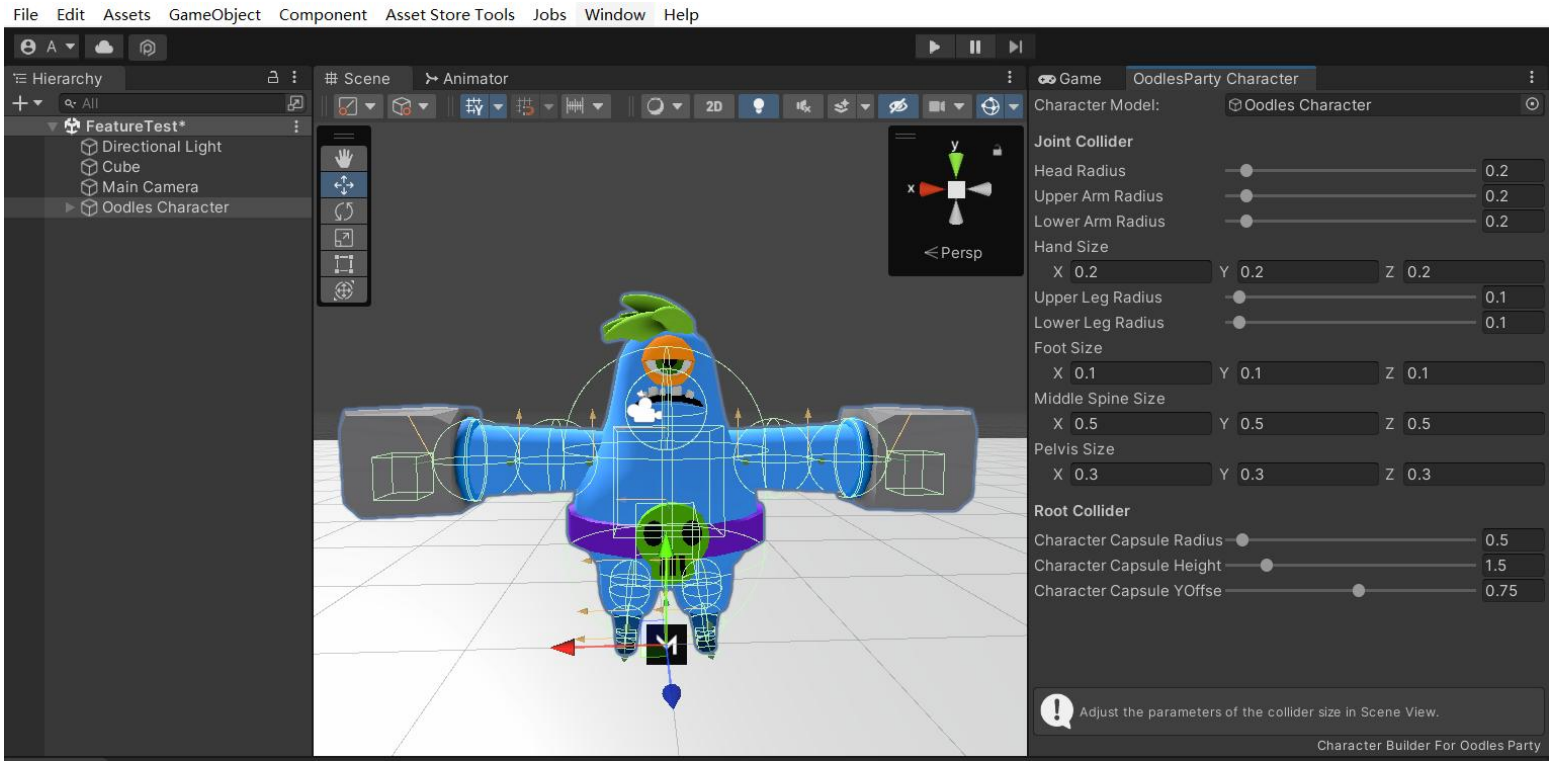

Open the Window -> OodlesCharacter panel.



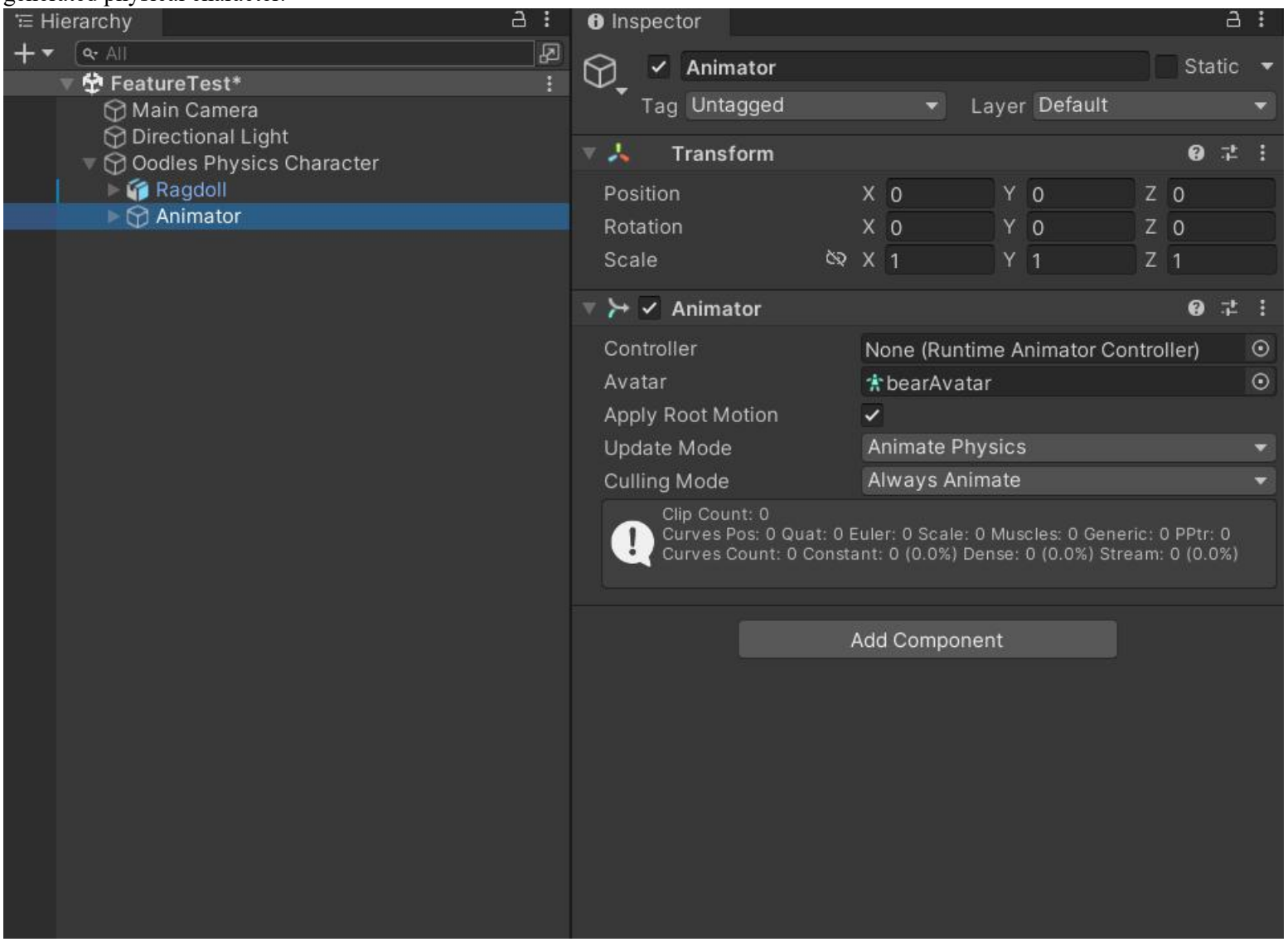

Next, select the character model in the Hierarchy.




Click the 'Create Character' button, and you will see the generated physical character.

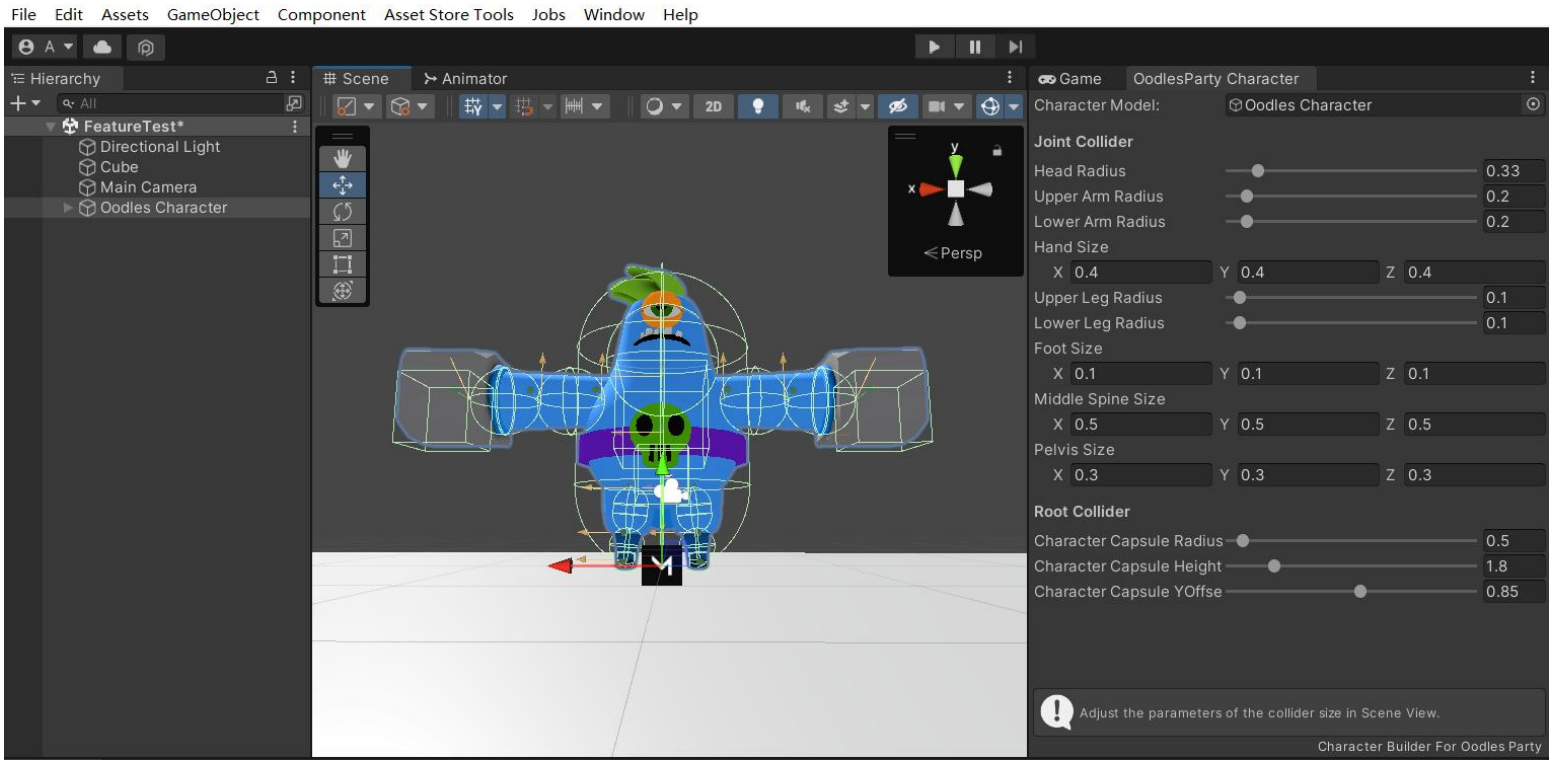



Finally, manually drag the **Assets/TechFusion/OodlesEngine/Animators/OodlesController** into the Animator component of the generated physical character.




## 2.2 Joint Bounding Box

The default parameters may not be suitable for your model size. In this case, select the newly generated physical character, compare the size of the physics bounding box with the Scene View, and adjust the parameters on the OodlesEngine Character panel to match the bounding box with the model.




Tips:

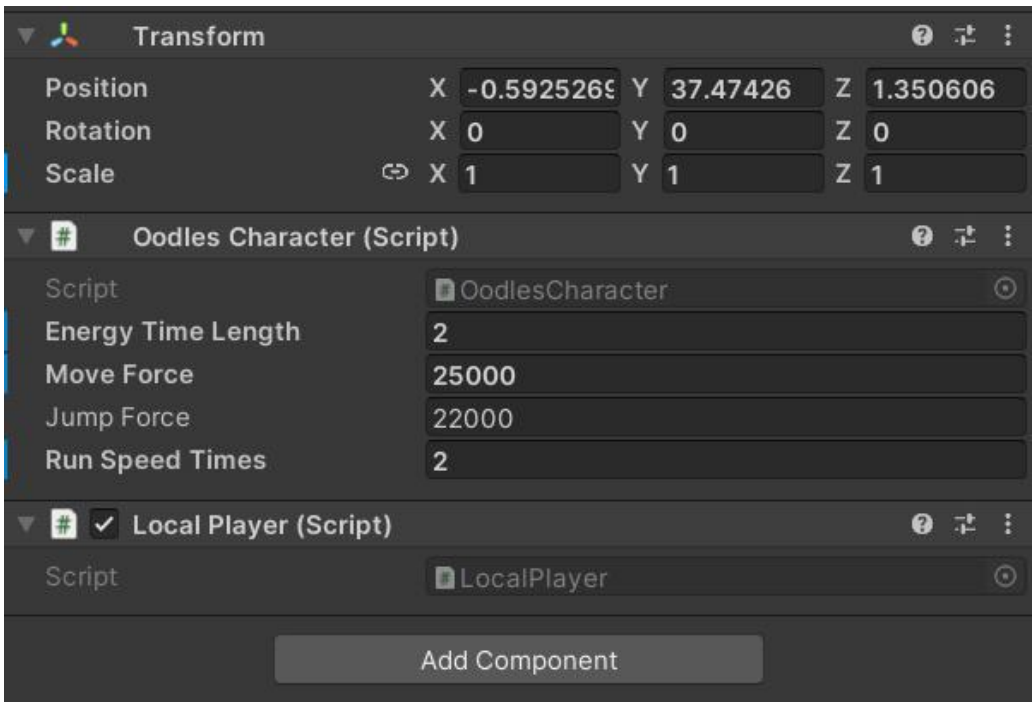
1.The Root Collider is a collider that encompasses the entire character. Please adjust the Radius and Height to precisely cover the dimensions of the character. It is recommended to set the Character Capsule Y Offset slightly less than half of the Character Capsule Height.

2.It is advisable to slightly reduce the size of the bounding boxes for the legs and feet. This can help avoid distortion in actions caused by the bounding boxes being too large and squeezing against each other.

3.Ensure that the sizes of all bounding boxes are not too small to prevent them from getting stuck during physics calculations due to mutual nesting.

## 2.3 Place it in the game

Give a name you like to the newly generated physical character, and Attach the LocalPlayer Component to the Physics Character , then save it as a Prefab.




Then place this Prefab into your scene.

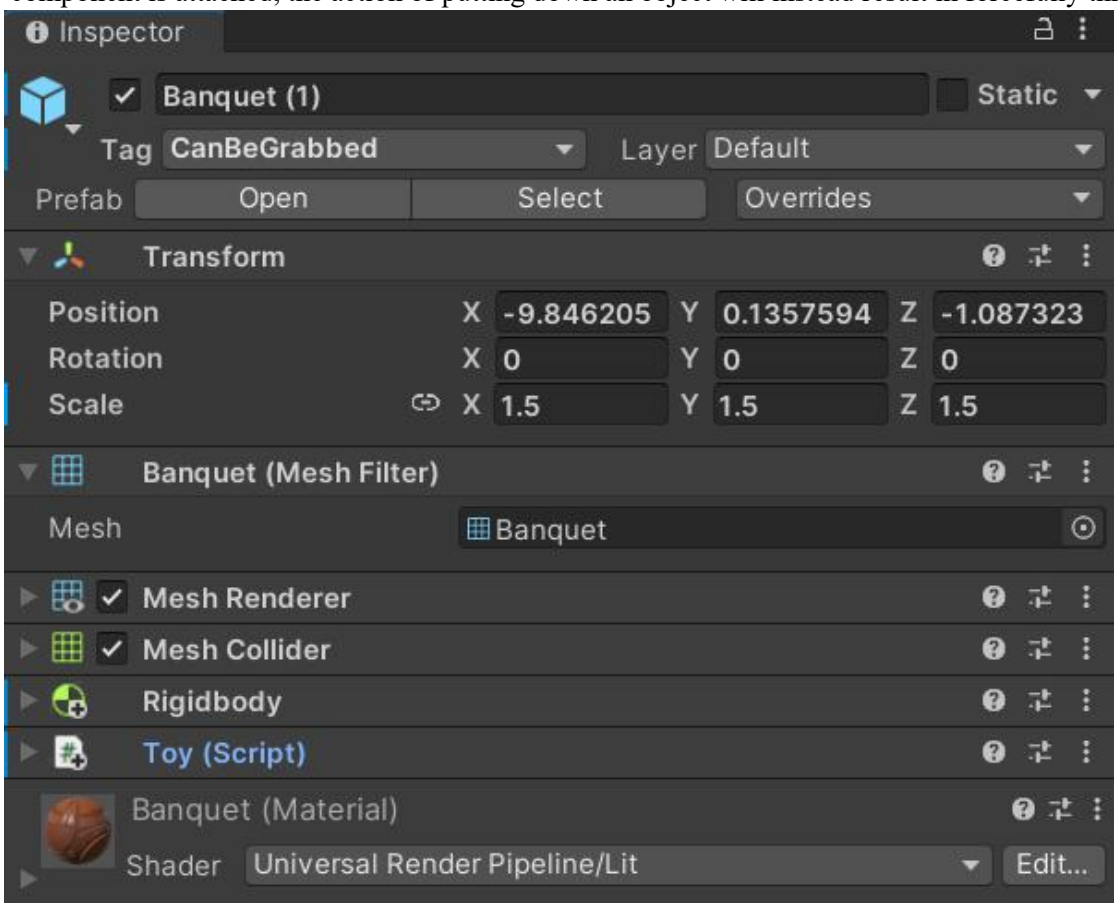
***Please note that in the current version, care should be taken to avoid any initial rotation when placing the character.**

# Part III: Creating Level

First and foremost, drag the **Assets/TechFusion/OodlesEngine/Prefabs/GameLocal.prefab** into the scene.

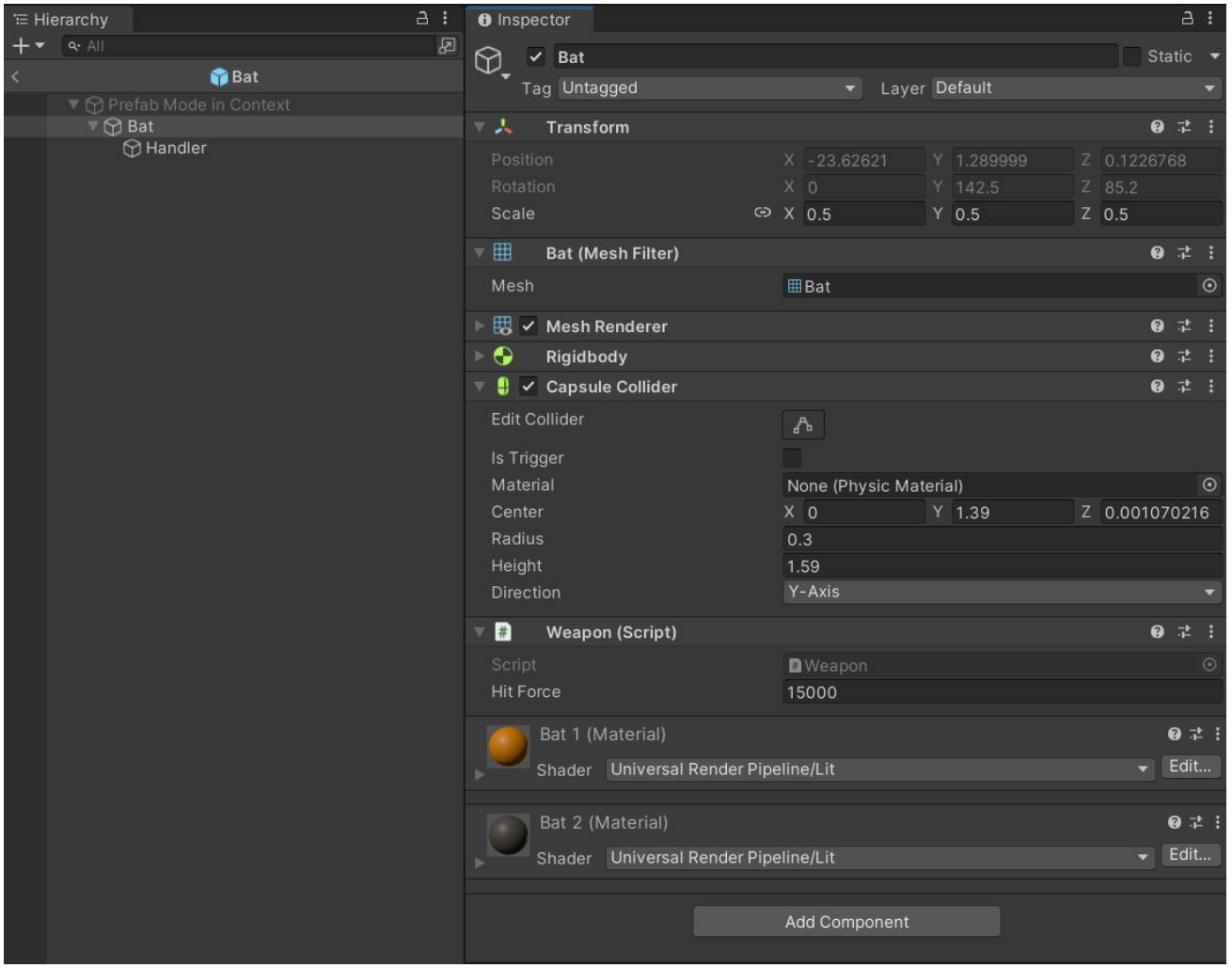
## 3.1 Grabbable objects

For objects that need to be picked up, you must attach both a Rigidbody and a Collider component, and they should be tagged with "CanBeGrabbed".
If a Toy component is attached, the action of putting down an object will instead result in forcefully throwing it.




## 3.2 Customizing Weapons

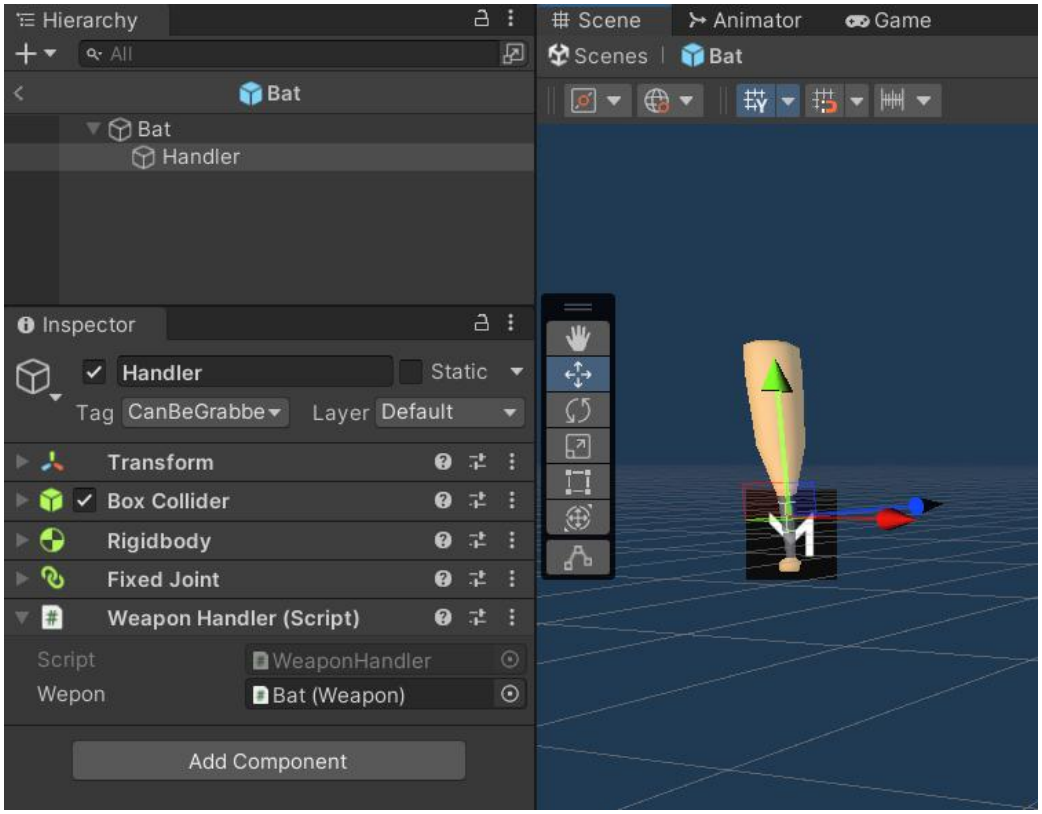
Configuring weapons is a bit more complex, as they can be picked up and moved by players.




Weapons require collision detection, so add a Rigidbody and Collider to the root GameObject. Then, add the Weapon.cs script component and set the Hit Force parameter to adjust the strength of the weapon.

Finally, create a child GameObject as the handle, responsible for collision detection with the player's hand. This GameObject should include:

- Rigidbody
- Collider
- Joint
- WeaponHandler.cs




## 3.3 Customizing Game Logic

The core game logic is implemented in **Assets/TechFusion/OodlesEngine/Scripts/Runtime/GamePlay/LocalGameManager.cs**. You can extend it by implementing these interfaces:

- public bool CanGrab(PlayerController pc, GameObject obj): Check if grabbing is allowed.
- public void OnGrabSomething(PlayerController pc, GameObject obj): Handle object grab events.
- public void OnReleaseSomething(PlayerController pc, GameObject obj): Handle object release events.
- public void OnThrowSomething(PlayerController pc, GameObject obj, Vector3 dir, bool twoHands): Handle object throwing events.
- public void OnHandAttackSomething(PlayerController pc, Collision col): Handle hand attack events.
- public void OnWeaponAttackSomething(PlayerController pc, Weapon wp, Vector3 dir, GameObject obj): Handle weapon attack events.

## You are now ready to expand and customize your game!